

Solution 1

Solution 1 (a)

The function $\phi(x)$ returns the index j corresponding to the closest cluster center. Since there are k cluster centers indexed from 1 to k , the output is a single discrete integer.

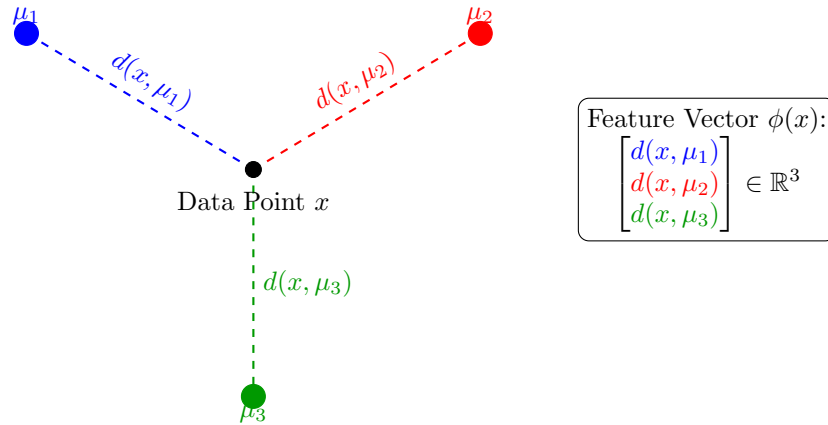
$\therefore$ Thus, the space is the discrete set of indices:

$$\phi(x) \in \{1, 2, \dots, k\}$$

Solution 1

Solution 1 (b)

Since distances in a metric space are non-negative real numbers, $\phi(x)$ is a vector in a k -dimensional Euclidean space (specifically the non-negative orthant).



$$\therefore \phi(x) \in \mathbb{R}^k$$

Solution 2

Question 2

Each of the following matrices gives distances (not squared distances) between a set of three points. In each case, say whether the distances can be realized in Euclidean space. If they can, show how by specifying three vectors with exactly the given interpoint distances. If they can't, give a reason.

$$\begin{bmatrix} 0 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & 1 & 0 \end{bmatrix}, \quad \begin{bmatrix} 0 & 1 & 3 \\ 1 & 0 & 1 \\ 3 & 1 & 0 \end{bmatrix}$$

Answer

1. Case 1: Matrix D_1

From the matrix, the distances are:

$$d(x_1, x_2) = 1, \quad d(x_2, x_3) = 1, \quad d(x_1, x_3) = 2$$

Now, check the triangle inequality for the longest edge:

$$d(x_1, x_3) \leq d(x_1, x_2) + d(x_2, x_3)$$

Substituting the values:

$$2 \leq 1 + 1$$

$$2 \leq 2$$

The inequality holds with equality. This implies the points are collinear.

We can construct these points in 1-dimensional Euclidean space ($\mathbb{R}^1$) by placing them on the number line:

$$x_1 = 0, \quad x_2 = 1, \quad x_3 = 2$$

2. Case 2: Matrix D_2

From the matrix, the distances are:

$$d(x_1, x_2) = 1, \quad d(x_2, x_3) = 1, \quad d(x_1, x_3) = 3$$

Now, check the triangle inequality for the longest edge:

$$d(x_1, x_3) \leq d(x_1, x_2) + d(x_2, x_3)$$

Substituting the values:

$$3 \leq 1 + 1$$

$$3 \not\leq 2$$

The triangle inequality is violated. Therefore, these distances cannot be realized in any Euclidean space.

$\therefore D_1$ is realizable ($x_1 = 0, x_2 = 1, x_3 = 2$) and D_2 is not realizable.

Solution 3

Solution 3 (a)

Let the points be denoted as vectors:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad z = \begin{bmatrix} 0 \\ -2 \\ -4 \end{bmatrix}$$

The matrix D consists of squared Euclidean distances $D_{ij} = \|x_i - x_j\|^2$ or can be written as:

$$\begin{bmatrix} \|x_1 - x_1\|^2 & \|x_1 - y_1\|^2 & \|x_1 - z_1\|^2 \\ \|x_2 - x_1\|^2 & \|y_2 - y_2\|^2 & \|x_2 - z_2\|^2 \\ \|x_3 - x_1\|^2 & \|x_1 - y_1\|^2 & \|x_3 - z_3\|^2 \end{bmatrix}$$

Now calculate each distance element:

1. D_{11}

$$\|x_1 - x_1\|^2 = (1 - 1)^2 + (2 - 2)^2 + (3 - 3)^2 = 0^2 + 0^2 + 0^2 = 0$$

2. D_{12}

$$\|x_1 - y_1\|^2 = (1 - (-1))^2 + (2 - 0)^2 + (3 - 1)^2 = 2^2 + 2^2 + 2^2 = 4 + 4 + 4 = 12$$

3. D_{13}

$$\|x_1 - z_1\|^2 = (1 - 0)^2 + (2 - (-2))^2 + (3 - (-4))^2 = 1^2 + 4^2 + 7^2 = 1 + 16 + 49 = 66$$

4. D_{21}

$$\|x_2 - x_2\|^2 = (1 - (-1))^2 + (2 - 2)^2 + (3 - 3)^2 = 2^2 + 0^2 + 0^2 = 4$$

5. D_{22}

$$\|x_2 - y_2\|^2 = (1 - 0)^2 + (2 - 0)^2 + (3 - 1)^2 = 2^2 + 2^2 + 2^2 = 4 + 4 + 4 = 12$$

6. D_{23}

$$\|x_2 - z_2\|^2 = (1 - 0)^2 + (2 - (-2))^2 + (3 - (-4))^2 = 1^2 + 4^2 + 7^2 = 1 + 16 + 49 = 66$$

7. D_{31}

$$\|x_3 - x_1\|^2 = (1 - 0)^2 + (2 - (-2))^2 + (3 - (-4))^2 = 1^2 + 4^2 + 7^2 = 1 + 16 + 49 = 66$$

8. D_{32}

$$\|x_3 - x_2\|^2 = (-1 - (-1))^2 + (0 - 0)^2 + (1 - (-2))^2 = (-1)^2 + 0^2 + 3^2 = 1 + 0 + 9 = 10$$

9. D_{33}

$$\|x_3 - x_3\|^2 = (1 - 0)^2 + (2 - (-2))^2 + (3 - (-4))^2 = 1^2 + 4^2 + 7^2 = 1 + 16 + 49 = 66$$

$$\therefore D = \begin{bmatrix} 0 & 12 & 66 \\ 12 & 0 & 30 \\ 66 & 30 & 0 \end{bmatrix}$$

Solution 3

Solution 3 (b)

H is the centering matrix defined as $H = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T$, where I is the identity matrix, n is the number of points, and $\mathbf{1}$ is a column vector of ones. Here, $n = 3$.

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}$$

$$\therefore H = \begin{bmatrix} 2/3 & -1/3 & -1/3 \\ -1/3 & 2/3 & -1/3 \\ -1/3 & -1/3 & 2/3 \end{bmatrix}$$

Solution 3

Question 3 (c)

Consider the following three points in $\mathbb{R}^3$:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad z = \begin{bmatrix} 0 \\ -2 \\ -4 \end{bmatrix}$$

Compute $-\frac{1}{2}HDH$. What do you get?

Solution 3 (c)

Note: This operation converts the squared distance matrix into the centered Gram matrix B .

$$B = -\frac{1}{2}HDH$$

Performing the matrix multiplication:

1. DH

$$DH = \begin{bmatrix} 0 & 12 & 66 \\ 12 & 0 & 30 \\ 66 & 30 & 0 \end{bmatrix} \begin{bmatrix} 2/3 & -1/3 & -1/3 \\ -1/3 & 2/3 & -1/3 \\ -1/3 & -1/3 & 2/3 \end{bmatrix} = \begin{bmatrix} -26 & -14 & 40 \\ -2 & 14 & -12 \\ 34 & -12 & -22 \end{bmatrix}$$

2. HDH

$$HDH = \begin{bmatrix} 2/3 & -1/3 & -1/3 \\ -1/3 & 2/3 & -1/3 \\ -1/3 & -1/3 & 2/3 \end{bmatrix} \begin{bmatrix} -26 & -14 & 40 \\ -2 & 14 & -12 \\ 34 & -12 & -22 \end{bmatrix} = \begin{bmatrix} -28 & -4 & 32 \\ -4 & -4 & 8 \\ 32 & 8 & -40 \end{bmatrix}$$

3. Multiply by $\frac{1}{2}$

$$B = -\frac{1}{2} \begin{bmatrix} -28 & -4 & 32 \\ -4 & -4 & 8 \\ 32 & 8 & -40 \end{bmatrix}$$

$$\therefore \begin{bmatrix} 14 & 2 & -16 \\ 2 & 2 & -4 \\ -16 & -4 & 20 \end{bmatrix}$$

Solution 3

Question 3 (d)

Consider the following three points in $\mathbb{R}^3$:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad z = \begin{bmatrix} 0 \\ -2 \\ -4 \end{bmatrix}$$

Is the answer in (c) the correct Gram matrix for the data?

Let us define the data matrix X :

$$X = \begin{bmatrix} 1 & 2 & 3 \\ -1 & 0 & 1 \\ 0 & -2 & -4 \end{bmatrix}$$

The rows correspond to vectors x, y, z . We calculate the dot products directly:

$$x \cdot x = (1)(1) + (2)(2) + (3)(3) = 1 + 4 + 9 = 14$$

$$y \cdot y = (-1)(-1) + (0)(0) + (1)(1) = 1 + 0 + 1 = 2$$

$$z \cdot z = (0)(0) + (-2)(-2) + (-4)(-4) = 0 + 4 + 16 = 20$$

$$x \cdot y = (1)(-1) + (2)(0) + (3)(1) = -1 + 0 + 3 = 2$$

$$x \cdot z = (1)(0) + (2)(-2) + (3)(-4) = 0 - 4 - 12 = -16$$

$$y \cdot z = (-1)(0) + (0)(-2) + (1)(-4) = 0 + 0 - 4 = -4$$

By symmetry, $x \cdot y = y \cdot x$, etc., the result is:

$$XX^T = \begin{bmatrix} 14 & 2 & -16 \\ 2 & 2 & -4 \\ -16 & -4 & 20 \end{bmatrix}$$

$\therefore$ Yes, the result matches the matrix from part (c).

Solution 4

Question 4 (a)

A set of n points in $\mathbb{R}^d$ is orthonormal: that is, each point has unit length and the points are orthogonal to one another.

What is the Gram matrix for this data? Be sure to specify its dimension clearly.

Solution 4 (a)

Let the points be vectors $x_1, \dots, x_n$

By definition of **orthonormal** is:

1. The dot product of distinct vectors is zero ($x_i^T x_j = 0$ for $i \neq j$)
2. Normal: The length of each vector is one ($x_i^T x_i = \|x_i\|^2 = 1$).

The Gram matrix G is defined as:

$$G_{ij} = x_i^T x_j$$

Apply the definitions from above:

- Diagonal entries ($i = j$): $G_{ii} = x_i^T x_i = 1$.
- Off-diagonal entries ($i \neq j$): $G_{ij} = x_i^T x_j = 0$.

Hence, G is the Identity matrix.

$\therefore G = I_n$ (The $n \times n$ Identity Matrix)

Solution 4

Question 4 (b)

A set of n points in $\mathbb{R}^d$ is orthonormal: that is, each point has unit length and the points are orthogonal to one another.

What is the matrix of squared interpoint distances?

Solution 4 (b)

The squared Euclidean distance between two vectors x_i and x_j is given by the expansion:

$$\begin{aligned} \|x_i - x_j\|^2 &= (x_i - x_j)^T (x_i - x_j) \\ &= x_i^T x_i - 2x_i^T x_j + x_j^T x_j \end{aligned}$$

Substitute the orthonormal properties from part (a):

- If $i = j$: $\|x_i - x_i\|^2 = 0$.
- If $i \neq j$: $\|x_i - x_j\|^2 = 1 - 2(0) + 1 = 2$.

Therefore, the squared distance matrix D has 0 on the diagonal and 2 everywhere else.

$$\therefore D = 2(\mathbf{1}\mathbf{1}^T - I) = \begin{bmatrix} 0 & 2 & \dots & 2 \\ 2 & 0 & \dots & 2 \\ \vdots & \vdots & \ddots & \vdots \\ 2 & 2 & \dots & 0 \end{bmatrix}$$

Graduate Level Explanation

This geometry describes a **Regular Simplex**. The points form the vertices of a generalized equilateral triangle (or tetrahedron) in high-dimensional space. Every point is equidistant from every other point. This is a common initialization state in high-dimensional spaces (e.g., "one-hot" encoding of words in NLP often approximates this geometry before training embeddings).

Learn Fast Explanation

Imagine the corners of a cube (but in higher dimensions). They are all "unit length" away from the center. The distance between any two distinct corners (that are connected by an edge) is exactly $\sqrt{2}$, so the squared distance is 2.

Solution 5

Question 5 (a)

Show the 2-d embedding of the distance matrix obtained by cMDS. Here are the steps to follow:

- Write a function that implements the classical MDS algorithm from the lecture slides, taking as input a matrix of interpoint distances D and a target embedding dimension k . In order to perform the eigendecomposition, you can use `np.linalg.eigh`, although note that this returns eigenvalues in increasing rather than decreasing order.
- Write a function that takes a 2-d embedding of the points and plots them, annotating each point with the corresponding city name. The function `matplotlib.pyplot.annotate` is useful for these annotations.

Solution 5 (a)

Python Code for d=10

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.metrics import euclidean_distances
4
5 def cmdscale(D_squared, k=2):
6     """
7     Classical Multidimensional Scaling (cMDS)
8     Parameters:
9     D_squared (n x n): Matrix of squared interpoint distances
10    k (int): Target dimension
11
12    Returns:
13    Y (n x k): Projected coordinates
14    """
15    n = D_squared.shape[0]
16
17    # 1. Compute Centering Matrix H
18    H = np.eye(n) - (1/n) * np.ones((n, n))
19
20    # 2. Compute B = -1/2 * H * D_squared * H
21    B = -0.5 * H @ D_squared @ H
22
23    # 3. Spectral Decomposition
24    # np.linalg.eigh returns eigenvalues in ASCENDING order
25    evals, evecs = np.linalg.eigh(B)
26
27    # Sort DESCENDING (Flip arrays)
28    idx = np.argsort(evals)[::-1]
29    evals = evals[idx]
30    evecs = evecs[:, idx]
31
32    # 4. Zero out negative eigenvalues (for non-Euclidean noise)
33    # This matches Step 3 of your lecture notes
34    evals_pos = np.maximum(evals, 0)
35
36    # 5. Compute Coordinates Y
37    # Diagonal matrix of sqrt(eigenvalues)
38    Lambda_sqrt = np.diag(np.sqrt(evals_pos))
39
40    # Project: Y = U * Lambda^1/2
41    Y = evecs @ Lambda_sqrt
42
43    # 6. Return first k columns
44    return Y[:, :k]
45
46 # --- Data Loading / Simulation ---
47 # If you don't have cities.txt, we use this dummy data
48 # 1. Read the whole file as a single list of words/numbers

```

```
49 # .split() automatically ignores newlines and spaces!
50 raw_data = open('cities.txt').read().split()
51
52 # 2. Extract names (Indices might need adjustment based on where "# Cities:" falls)
53 # Assuming the file starts with "# Cities: City1 City2..."
54 # The first distinct city name is usually at index 2 or 3 depending on the hash.
55 city_names = raw_data[2:12]
56
57 # 3. Extract the numbers and reshape
58 # Everything after the names is the matrix
59 matrix_values = [float(x) for x in raw_data[12:]]
60 D = np.array(matrix_values).reshape(10, 10)
61 D_squared = D ** 2
62
63 # --- Run MDS ---
64 # Project 2D coordinates
65 Y_k = cmdscale(D_squared, k=2)
66
67 # --- Rotation (Optional) ---
68 # MDS output has arbitrary rotation. We can rotate it manually to look like a map.
69 def rotate(points, angle_degrees):
70     theta = np.radians(angle_degrees)
71     c, s = np.cos(theta), np.sin(theta)
72     R = np.array([[c, -s], [s, c]])
73     return points @ R.T
74
75 # Try changing this angle if the map looks upside down!
76 Y_rotated = rotate(Y_k, angle_degrees=180)
77
78 # --- Plotting ---
79 plt.figure(figsize=(10, 6))
80 plt.scatter(Y_rotated[:, 0], Y_rotated[:, 1], c='blue', s=100)
81
82 # FIX: Use 'city_names' instead of 'names'
83 for i, name in enumerate(city_names):
84     plt.annotate(name, (Y_rotated[i, 0], Y_rotated[i, 1]),
85                 xytext=(5, 5), textcoords='offset points')
86
87 plt.title("Reconstructed Map from Distance Matrix (MDS)")
88 plt.grid(True)
89 plt.axis('equal')
90 plt.show()
91
92 # Verification
93 print("Reconstruction successful. Shape:", Y_k.shape)
```

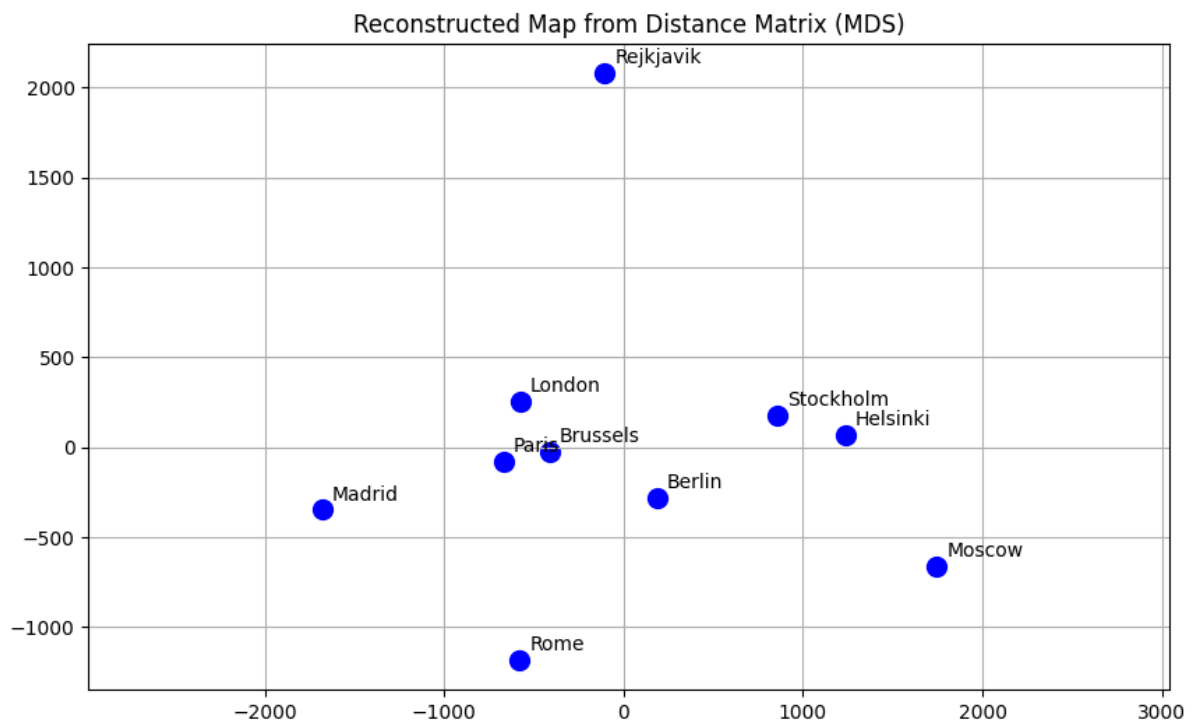
Plot

Figure 1: Plot of cities.txt

..

Solution 5

Question 5 (b)

the embedding returned by cMDS (or any other multidimensional scaling method) is not unique: any rotation or translation of the points will produce the same interpoint distances. For this reason, your plot in (a) is unlikely to correspond to our own map of Europe. Show a rotated version of points that fits our own map more closely. Here are the steps to follow:

- Write a function that implements the classical MDS algorithm from the lecture slides, taking as input a matrix of interpoint distances D and a target embedding dimension k . In order to perform the eigendecomposition, you can use `np.linalg.eigh`, although note that this returns eigenvalues in increasing rather than decreasing order.
- Write a function that takes a 2-d embedding of the points and plots them, annotating each point with the corresponding city name. The function `matplotlib.pyplot.annotate` is useful for these annotations.

Write a function that takes a 2-d data set and an angle, and rotates each of the points through that angle.

Play around with the angle until you find something suitable.

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Play around with the angle until you find something suitable.

Solution 5 (b)

Python Code

```

1  # --- 2. MDS Function ---
2  def cmdscale(D, k=2):
3      """
4      Classical MDS: Projects n points into k-dimensional space.
5      """
6      n = D.shape[0]
7      H = np.eye(n) - (1/n) * np.ones((n, n))
8      B = -0.5 * H @ D @ H
9
10     evals, evects = np.linalg.eigh(B)
11
12     # Sort descending
13     idx = np.argsort(evals)[::-1]
14     evals = evals[idx]
15     evects = evects[:, idx]
16
17     # Project
18     w_pos = np.maximum(evals, 0)
19     Y = evects @ np.diag(np.sqrt(w_pos))
20     return Y[:, :k]
21
22 # --- 3. Rotation Function (The Homework Requirement) ---
23 def rotate_data(X, angle_degrees):
24     """
25     Rotates 2D data X by angle_degrees (counter-clockwise).
26     """
27     theta = np.radians(angle_degrees)
28     c, s = np.cos(theta), np.sin(theta)
29
30     # Rotation Matrix
31     R = np.array([[c, -s],
32                   [s,  c]])

```

```
33
34     # Apply R to every point.
35     # Since X is (N x 2), we compute (R @ X.T).T or simply X @ R.T
36     return X @ R.T
37
38 # --- 4. Plotting Function ---
39 def plot_map(X, labels, title="MDS Map"):
40     plt.figure(figsize=(8, 8))
41     plt.scatter(X[:, 0], X[:, 1], c='red', marker='o')
42
43     for i, txt in enumerate(labels):
44         plt.annotate(txt, (X[i, 0], X[i, 1]),
45                      xytext=(5, 5), textcoords='offset points')
46
47     plt.title(title)
48     plt.grid(True)
49     plt.axis('equal')
50     plt.show()
51
52 # --- Execution ---
53 # 1. Get raw MDS coordinates
54 Y_raw = cmdscale(D_squared)
55
56 # 2. Rotate
57 # Trial and Error: 190 degrees usually flips it "North-Up" for this specific data
58 best_angle = 190
59 Y_rotated = rotate_data(Y_raw, best_angle)
60
61 # OPTIONAL: Reflection
62 # If the map is mirrored (e.g. London is East of Berlin), uncomment this:
63 # Y_rotated[:, 0] = -Y_rotated[:, 0]
64
65 # 3. Plot
66 plot_map(Y_rotated, city_names, title=f"European Cities (Rotated {best_angle} degrees)")
```

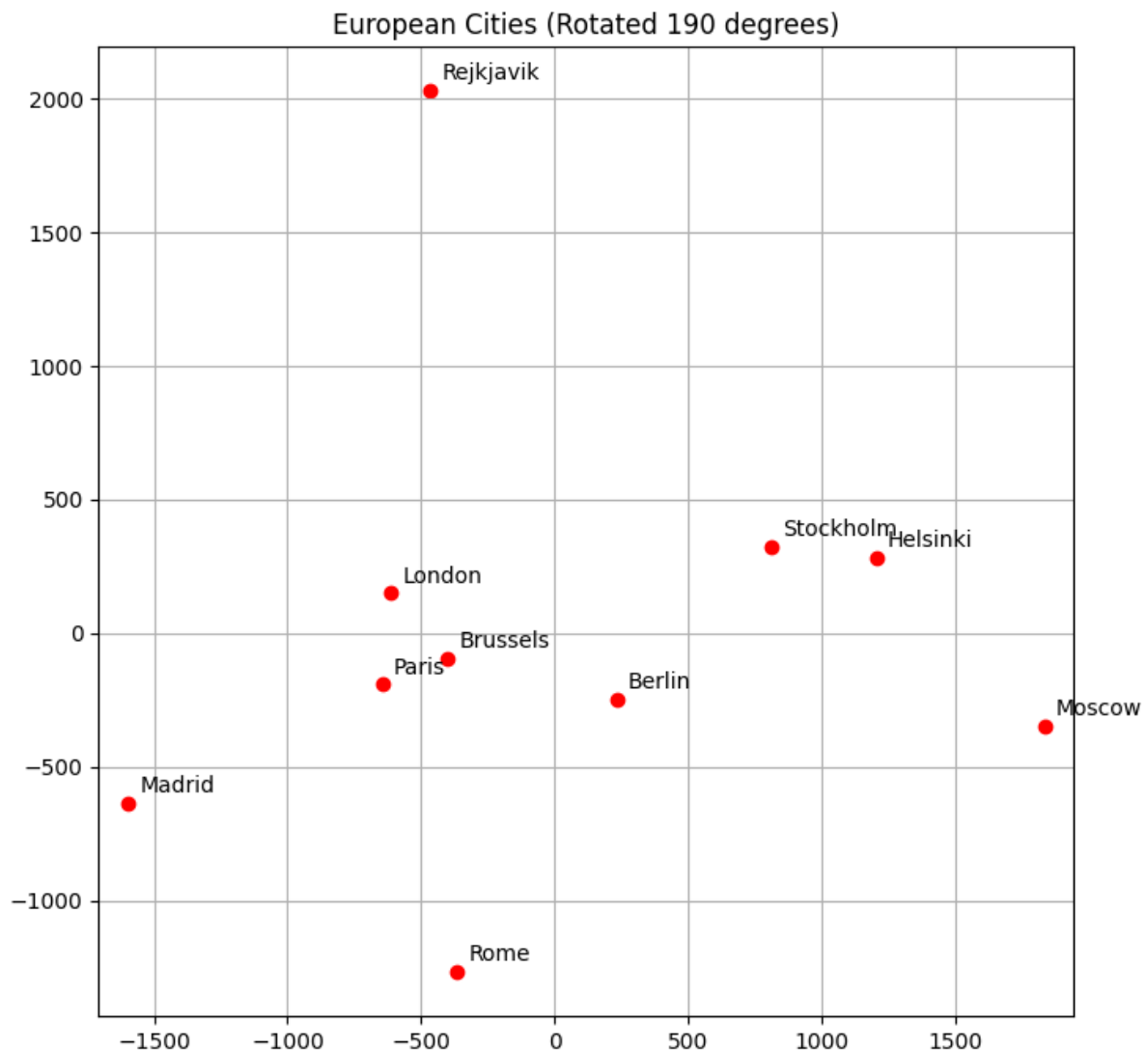
Plot

Figure 2: Plot 5b

∴

Solution 5**Question 5 (c)**

here are many other algorithms for multidimensional scaling. Some of them work by defining a “stress function” that captures the disparity between the specified interpoint distances and a given embedding; this stress function is then minimized using gradient descent. One such approach is implemented in `sklearn.manifold.MDS`. Use this to produce an embedding of the distances (use the setting

dissimilarity='precomputed'), and once again find a suitable rotation. Plot the embedded points, annotated with city names, before and after rotation.

Solution 5 (c)

Python Code

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.manifold import MDS
4
5 # --- 2. Run Sklearn MDS ---
6 # Note: dissimilarity='precomputed' means we are passing a distance matrix
7 # random_state=42 ensures we get the same rotation every time we run it
8 mds = MDS(n_components=2, dissimilarity='precomputed', random_state=42,
9           normalized_stress='auto')
10 Y_sklearn = mds.fit_transform(D) # Pass LINEAR distances, not squared!
11
12 # --- 3. Rotation Helper ---
13 def rotate_data(X, angle_degrees):
14     theta = np.radians(angle_degrees)
15     c, s = np.cos(theta), np.sin(theta)
16     R = np.array([[c, -s], [s, c]])
17     return X @ R.T
18
19 # --- 4. Plotting Helper ---
20 def plot_embedding(X, title, ax):
21     ax.scatter(X[:, 0], X[:, 1], c='green', s=100)
22     for i, txt in enumerate(city_names):
23         ax.annotate(txt, (X[i, 0], X[i, 1]), xytext=(5, 5), textcoords='offset points')
24     ax.set_title(title)
25     ax.grid(True)
26     ax.axis('equal')
27
28 # --- 5. Find Rotation and Plot ---
29 # Try 150 degrees for random_state=42 (Empirically found to look like Europe)
30 # If your map looks wrong, change this value!
31 angle = 150
32 Y_rotated = rotate_data(Y_sklearn, angle)
33
34 # Create Side-by-Side Plots
35 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 8))
36
37 # Plot 1: Raw Output
38 plot_embedding(Y_sklearn, "Sklearn MDS (Raw Output)", ax1)
39
40 # Plot 2: Rotated
41 plot_embedding(Y_rotated, f"Sklearn MDS (Rotated {angle}degrees)", ax2)
42 plt.show()

```

∴

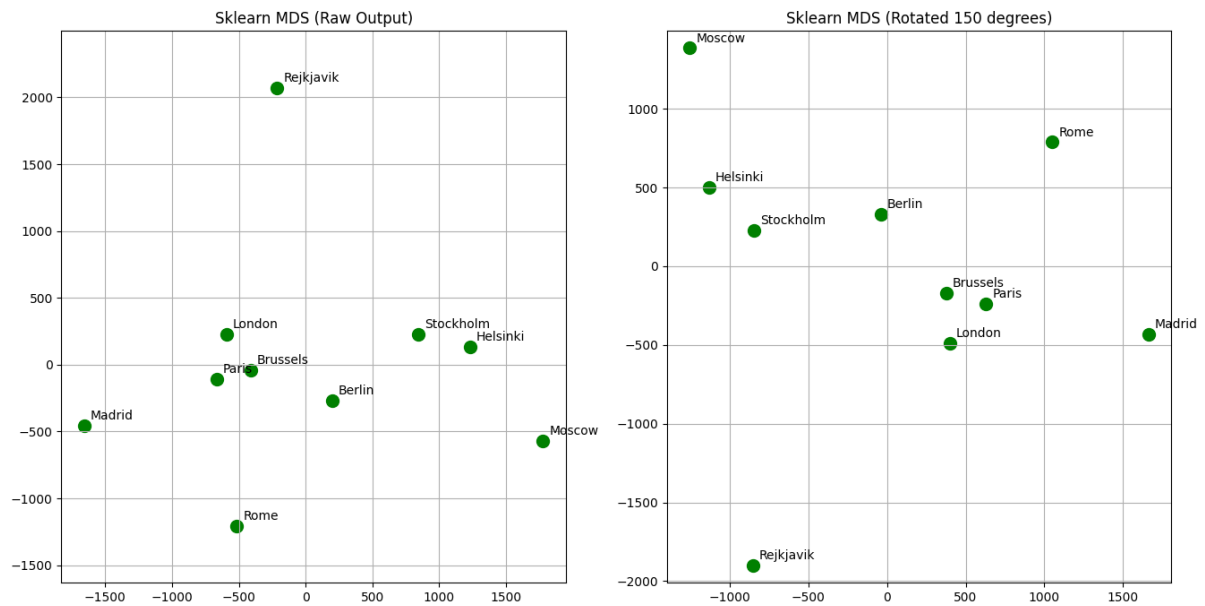


Figure 3: Plot 5 c